SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA1003	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	ALIGETI AKSHAYA	**Reg. No.:**	192425170
Date:	20-08-2026	**Duration:**	60 minutes

Code of Conduct

I __A.Akshaya(192425170)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _Akshaya_____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

ACADEMIC PROJECT, ALLOCATION MONITORING AND EVALUATION MANAGEMENT SYSTEM

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.

   The **Academic Project, Allocation Monitoring and Evaluation Management System** is a full-stack application containing a frontend, backend, database, authentication, project allocation, progress monitoring, and evaluation modules. Since different features may be developed and updated regularly, a CI/CD pipeline is useful to automatically build, test, check, and deploy the application.

   Main CI/CD Needs

- Automatically check the code whenever a developer pushes changes.
- Build the frontend and backend automatically.
- Run unit and integration tests.

- Check code quality and coding errors.
- Scan the application for security vulnerabilities.
- Create Docker images for deployment.
- Deploy the application to development and testing environments.
- Deploy the tested version to production.
- Notify the development team about successful or failed builds.
- Provide rollback when a newly deployed version causes problems.

2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.

   Developer

   ↓

   GitHub Repository

   ↓

   Code Commit / Pull Request

   ↓

   Build

   ↓

   Automated Testing

   ↓

   Code Quality Analysis

   ↓

   Security Scan

   ↓

   Docker Image Creation

   ↓

   Staging Deployment

↓

Integration / Acceptance Testing

↓

Approval

↓

Production Deployment

↓

Monitoring

↓

Rollback if Required

Pipeline Stages

1. Source Code Management

Developers push their code to GitHub using feature branches.

2. Build

The pipeline installs dependencies and builds the frontend and backend.

3. Automated Testing

Unit, integration, and API tests are executed automatically.

4. Code Quality Analysis

Tools check the source code for bugs, code smells, duplication, and maintainability issues.

5. Security Analysis

Dependencies and application code are scanned for known vulnerabilities.

6. Packaging

The frontend and backend are packaged into Docker images.

7. Staging Deployment

The application is deployed to a testing/staging environment.

8. Production Deployment

After successful testing and approval, the application is deployed to production.

3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.

Pipeline Stage	Tool	Purpose
Source Code	Git	Version control
Repository	GitHub	Store and collaborate on code
CI/CD	GitHub Actions	Automate the pipeline
Frontend	React / HTML / CSS / JavaScript	User interface
Backend	Node.js + Express.js	Server and APIs
Database	MySQL	Store application data
Testing	Jest	Unit testing
API Testing	Supertest / Postman	Test backend APIs
Code Quality	SonarQube	Analyze code quality
Security	npm audit	Check dependency vulnerabilities
Containerization	Docker	Package application
Container Management	Docker Compose	Run multiple services
Deployment	Docker / Cloud platform	Deploy application
Notifications	GitHub Actions notifications	Inform team about pipeline status

Why GitHub Actions?

GitHub Actions is suitable because the project code is already maintained using Git and can be stored in GitHub. It allows the CI/CD pipeline to automatically start whenever code is pushed or a pull request is created.

Why Docker?

Docker provides the same application environment during development, testing, and deployment. It can package the frontend, backend, and required dependencies into containers.

4. Define appropriate **automated testing strategies** to be integrated into the pipeline.

Automated testing should be included in the pipeline so that errors are detected before deployment.

Unit Testing

Individual functions and modules can be tested.

Examples:

- Login validation.
- Student profile validation.
- Project allocation logic.
- Marks calculation.
- Deadline validation.

Integration Testing

Integration testing checks whether different components work together correctly.

Examples:

- Frontend communicating with backend.
- Backend communicating with MySQL.
- Login API communicating with authentication system.
- Project allocation API storing data in the database.

API Testing

The backend APIs can be tested automatically.

Examples:

POST /login

POST /students

GET /projects

POST /allocation

POST /evaluation

GET /reports

Security Testing

The pipeline can check:

- Invalid login attempts.
- Unauthorized access.
- Role-based access.
- Vulnerable dependencies.
- Improper input validation.

Testing Strategy

Code Commit

↓

Unit Tests

↓

Integration Tests

↓

API Tests

↓

Security Tests

↓

Deployment

5. Design the **deployment strategy**, including development, testing/staging, and production environments.

   The project can use three environments.

   ### Development Environment

   This environment is used by developers while creating and modifying features.

   Developer → Feature Branch → Development Environment

Examples:

- Login development.
- Student profile development.
- Project allocation development.

Testing/Staging Environment

After development, the application is deployed to staging for complete testing.

Testing includes:

- Functional testing.
- Integration testing.
- Security testing.
- User acceptance testing.

Production Environment

After successful testing and approval, the application is deployed to production.

```
Development
     ↓
Testing / Staging
     ↓
Approval
     ↓
Production
```

6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.

   Security

   Security checks should be performed automatically in the pipeline.

- Use HTTPS in production.
- Never store passwords in source code.
- Store secrets using GitHub Actions Secrets.
- Use environment variables for configuration.
- Run npm audit to identify vulnerable packages.

- Apply role-based access control.
- Validate user inputs.
- Scan Docker images for vulnerabilities where possible.

Quality Checks

Before deployment, the pipeline should check:

- Code formatting.
- Coding standards.
- Unit test results.
- Integration test results.
- Code coverage.
- Security vulnerabilities.
- Build success.

If critical checks fail, the pipeline should stop.

Notifications

The development team should receive notifications when:

- Build succeeds.
- Tests fail.
- Security scan fails.
- Deployment succeeds.
- Deployment fails.

For example:

✅ Build Successful

✅ Tests Passed

✅ Security Check Passed

🚀 Production Deployment Successful

Rollback Mechanism

If a newly deployed version has a serious problem, the previous stable version should be restored.

Example:

Version 1.0 → Stable

↓

Version 1.1 → Deployment

↓

Problem Detected

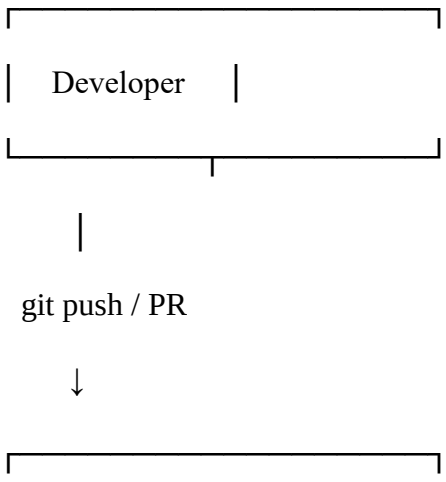
↓

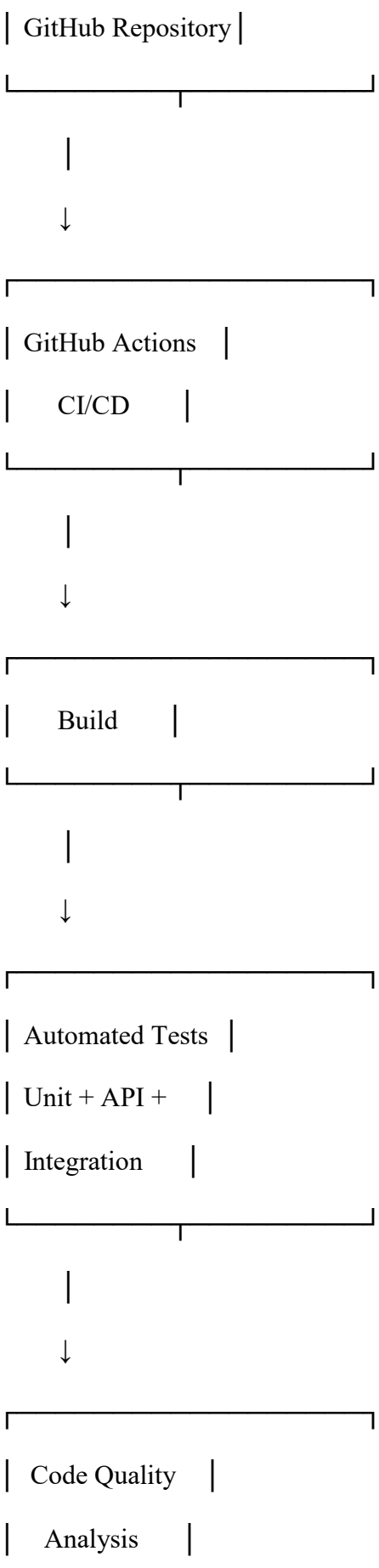
Rollback

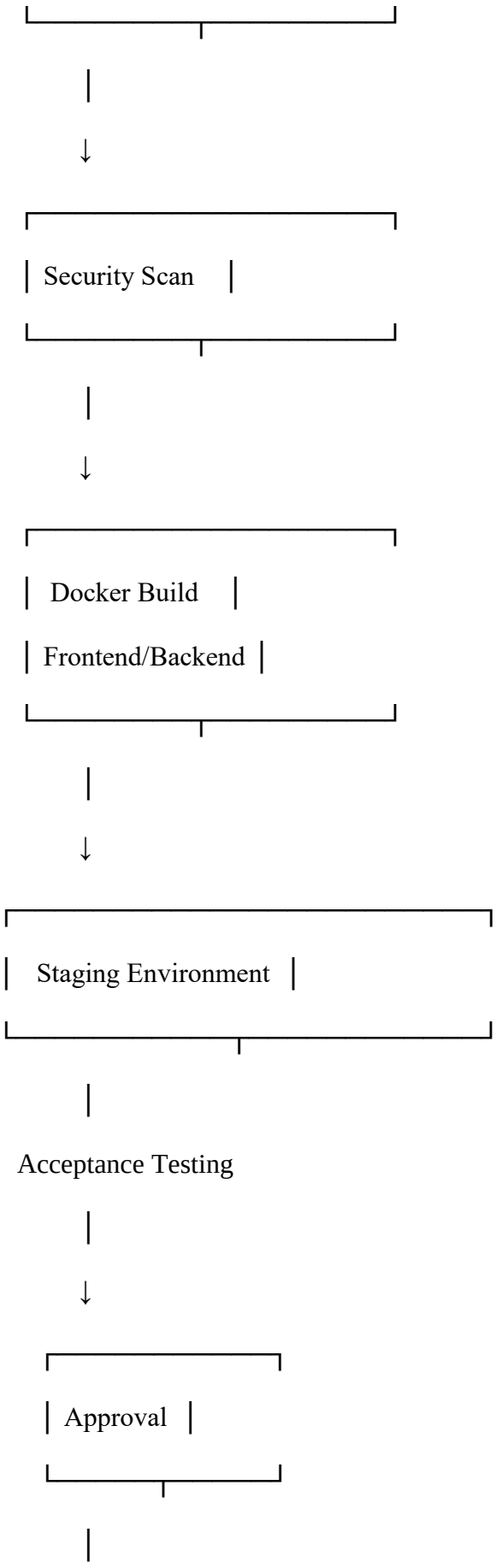
↓

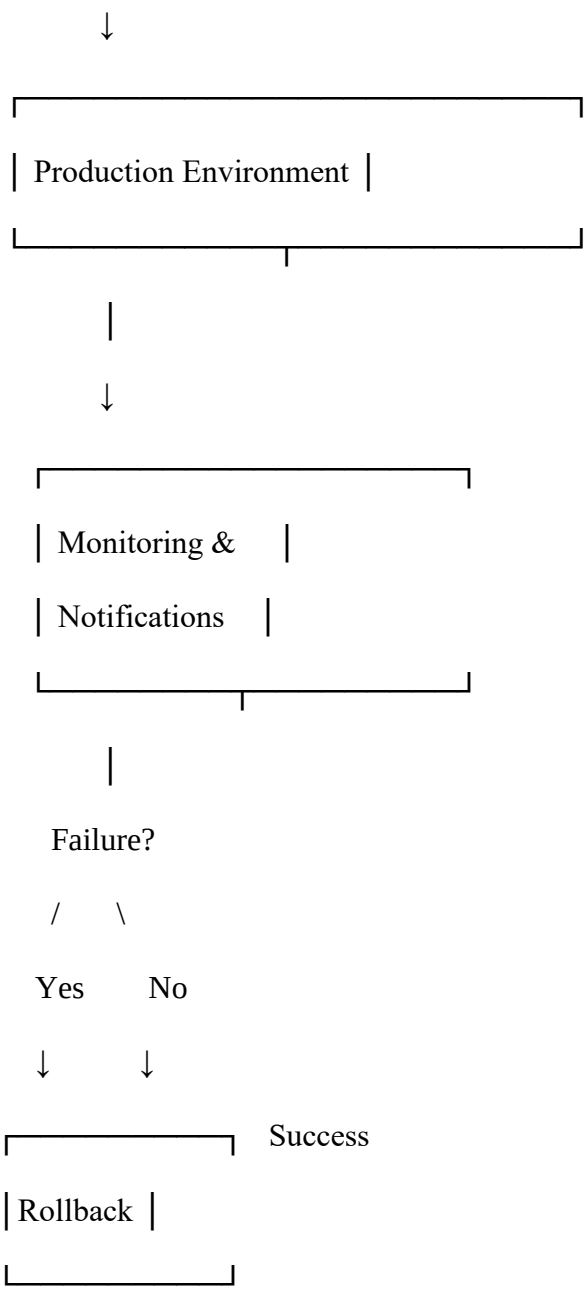
Version 1.0 → Restored

7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.




GitHub Repository
GitHub Actions
CI/CD
Build
Automated Tests
Unit + API +
Integration
Code Quality
Analysis


Security Scan
Docker Build
Frontend/Backend
Staging Environment
Acceptance Testing
Approval



Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollbac k details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/docum entation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4

Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25